

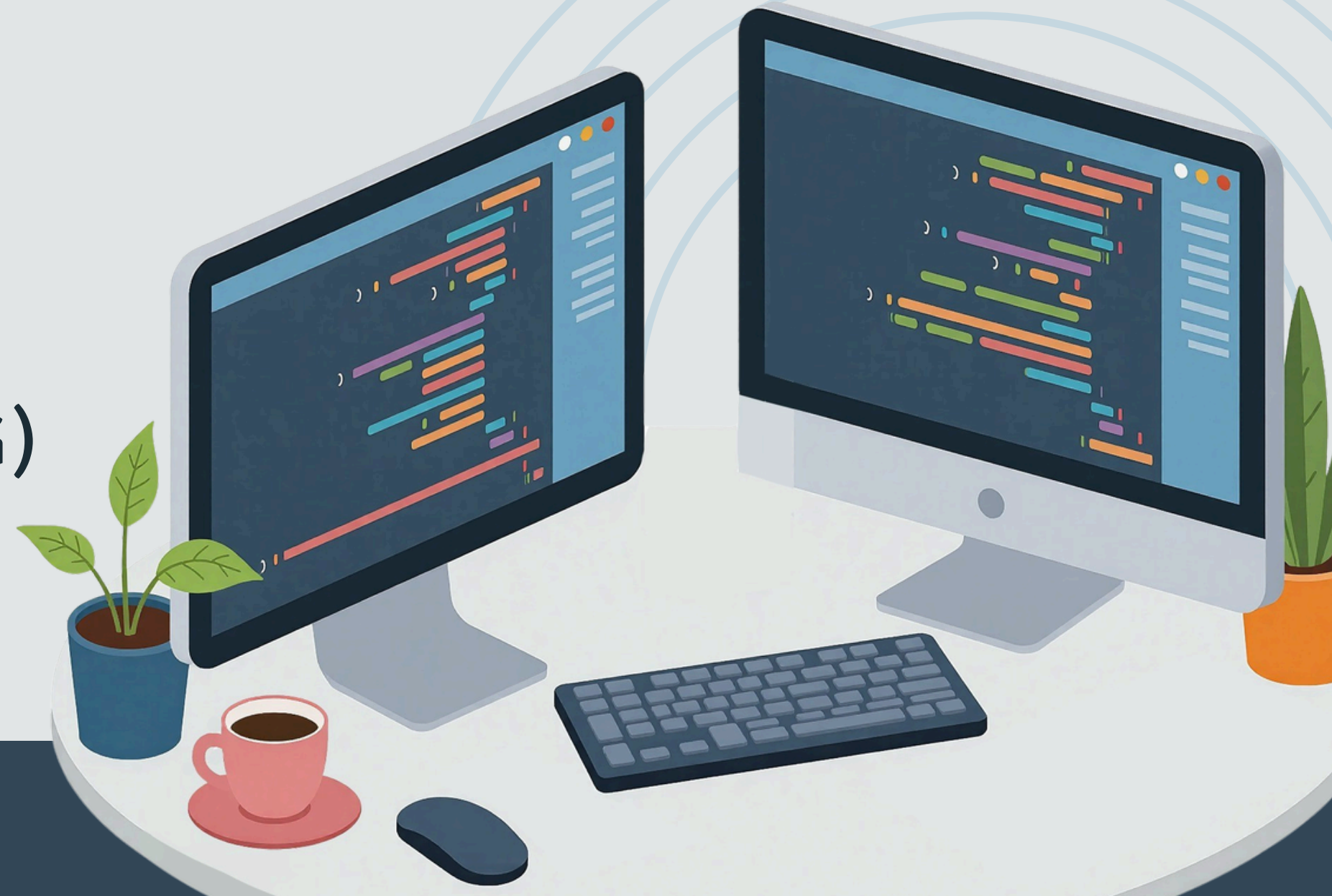
LAB 1

Presented by Team 1

AHMAD MOHAMMAD

ALFRED YAP KAI XIANG (U2520868G)

ASHA MEREDITH SOLOMON



Merge Sort

How it works?

- Divide the array into 2 halves
- Recursively sort both halves
- Merge the sorted halves

Time Complexity: $O(n \log n)$ - for best, average, and worst case

Space Complexity: $O(n)$ - temporary arrays are created during merging

Strength: Efficient & Predictable for large datasets

Weakness: Recursive calls & merging small subarrays will create extra overhead

Insertion Sort

How it works?

- Builds a sorted portion of the array one element at a time
- Inserts each new element into its correct position

Time Complexity: Best case: $O(n)$ - when array is sorted, Avg/Worst case: $O(n^2)$

Space Complexity: $O(1)$ - sorts in place

Strength: Simple, low overhead, good for small arrays

Weakness: Inefficient for large arrays



Problem

Merge sort is efficient for large amount of inputs, however when repeatedly splitting and making recursive calls will create unnecessary overhead.

Proposed Solution

Creating a Hybrid Merge Sort by combining **Merge Sort** and **Insertion Sort** and also introducing $S = \text{threshold}$.

Subarray size $> S$ - continue Merge Sort

Subarray $\leq S$ - switch to Insertion Sort



Objective

Implement and analyse a Hybrid Merge Sort that uses Insertion Sort for small subarrays, then determine whether it improves practical performance compared with original Merge Sort.

We will check:

1. How performance changes as input size n increases - c(i)
2. How performance changes as threshold S changes - c(ii)
3. How to choose an optimal S - c(iii)
4. Whether Hybrid Merge Sort outperforms the original Merge Sort - d



(a)Hybrid Merge Sort Pseudocode

HybridMergeSort(A, S):

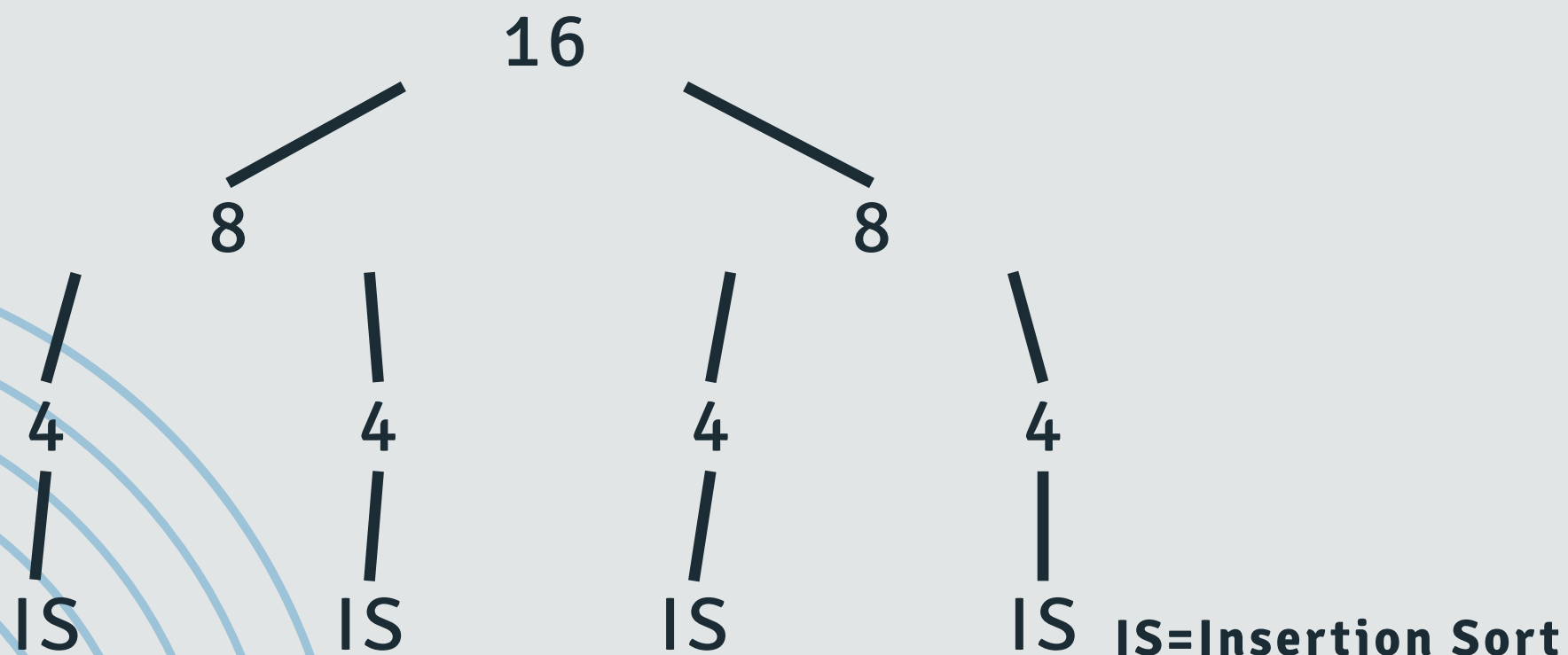
if size(A) <= S:
 use Insertion Sort

else:
 split A into left and right halves

HybridMergeSort(left, S)
HybridMergeSort(right, S)

merge the two sorted halves

Recursion Tree



```
def merge_hybrid_sort(arr, S=30):  
  
    # If array size is small enough, begins insertion sort instead  
    if (len(arr) <= S):  
        return insertionsort(arr)  
  
    # Split array and recursively sort each side  
    mid = len(arr) // 2  
    left = arr[:mid]  
    right = arr[mid:]  
  
    sorted_left = merge_hybrid_sort(left, S)  
    sorted_right = merge_hybrid_sort(right, S)  
  
    # Parses through sorted function output of [sorted_arr, comparison_count]  
    left = sorted_left[0]  
    right = sorted_right[0]  
    comparison_count = sorted_left[1] + sorted_right[1]  
  
    merged = np.empty(len(left) + len(right), dtype=arr.dtype)  
    i, j, k = 0, 0, 0  
  
    # While there are items remaining in both subarrays  
    while (i < len(left) and j < len(right)):  
        comparison_count += 1  
  
        # Compare which current item is greater and add it to the main array  
        if (left[i] <= right[j]):  
            merged[k] = left[i]  
            i += 1  
        else:  
            merged[k] = right[j]  
            j += 1  
  
        k += 1  
  
    # Add any remaining elements from either side  
    # No key-comparisons made here  
    if (i < len(left)):  
        merged[k:k + (len(left) - i)] = left[i:]  
  
    if (j < len(right)):  
        merged[k:k + (len(right) - j)] = right[j:]  
  
    return merged, comparison_count
```

Implementation & Experimental Setup

Functions	Purpose
merge_hybrid_sort()	Main hybrid algorithm
insertionsort()	Sorts small subarrays
merge_sort()	Standard Merge Sort baseline
_merge()	Merges two sorted halves
generate_array()	Generates random input data

Part (b) Dataset Generation

- $n = 1,000 \rightarrow 10,000,000$
- values: $1 \rightarrow 10,000,000$ (random value)
- seed = 42 (to get the same dataset everytime we run the program with this seed)

```
if (len(arr) <= S):  
    return insertionsort(arr)
```

 ---> condition that makes the algorithm hybrid

Key Pointers

- Using same dataset - to ensure fairness, the same datasets will be reused for each comparison; the original dataset will be copied before each sorting
- `time.process_time()` ---> measures how much time the sorting algorithm took (for part D)
- Key Comparision: a comparision between two data values/keys during sorting

Experiment	Fixed	Varied	Measured
C(i)	S	n	comparisions
C(ii)	n	S	comparisions
C(iii)	n	S	CPU+comp
D	10M	algorithm	CPU+Comp

1. C(i): How does the algorithm scale when input size grows?
2. C(ii): How does changing the threshold S affect comparisions?
3. C(iii): What value of S gives the best practical performance?
4. D: Does the hybrid algorithm actually outperforms standard Merge Sort?

C(i): Fixed S, Varying Input Size n

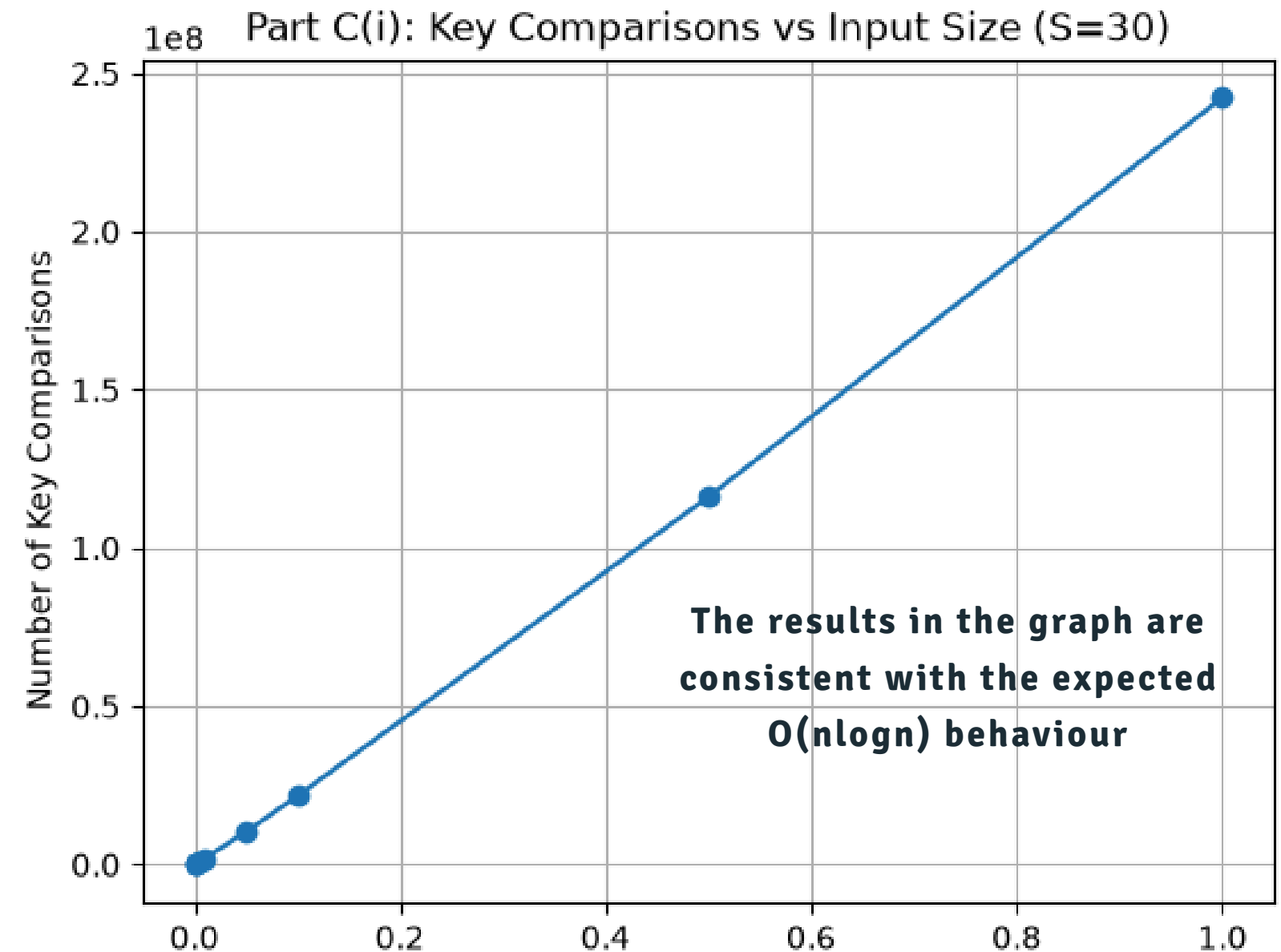
Set Up

- Fixed threshold: $S=30$
- Input size n : $1,000 \rightarrow 10,000,000$
- Random integer range: $[1, 10,000,000][1, 10,000,000]$
- Measure: Number of key comparisons + CPU time

Theoretical Expectation

Normal Merge Sort has time complexity: $O(n \log n)$, so by keeping S fixed at 30, the algorithm will still behave like a Merge Sort even when n increases. Therefore, we would expect a key comparisons to **increase apporximately with an $n \log n$ trend.**

n=	1,000	S=	30	comparisons=	10,318	CPU time=	0.0000s
n=	5,000	S=	30	comparisons=	67,043	CPU time=	0.0156s
n=	10,000	S=	30	comparisons=	143,479	CPU time=	0.0469s
n=	50,000	S=	30	comparisons=	880,922	CPU time=	0.2969s
n=	100,000	S=	30	comparisons=	1,865,125	CPU time=	0.6406s
n=	500,000	S=	30	comparisons=	10,299,949	CPU time=	3.6406s
n=	1,000,000	S=	30	comparisons=	21,593,998	CPU time=	7.7188s
n=	5,000,000	S=	30	comparisons=	116,163,473	CPU time=	43.7812s
n=	10,000,000	S=	30	comparisons=	242,317,231	CPU time=	90.1562s



Observations: As n increases, the number of key comparisons also increases (CPU time also increases).

The empirical results support the theoretical expectation: with S fixed, the hybrid algorithm shows Merge Sort-like $O(n \log n)$ growth.

C(ii): Fixed n, Varying S

Set Up

- Fixed n: $n=100,000$
- Varying S: 1,2,4,8,16,30,32,64,128
- Measure: Number of key comparisons + CPU time

Theoretical Expectation

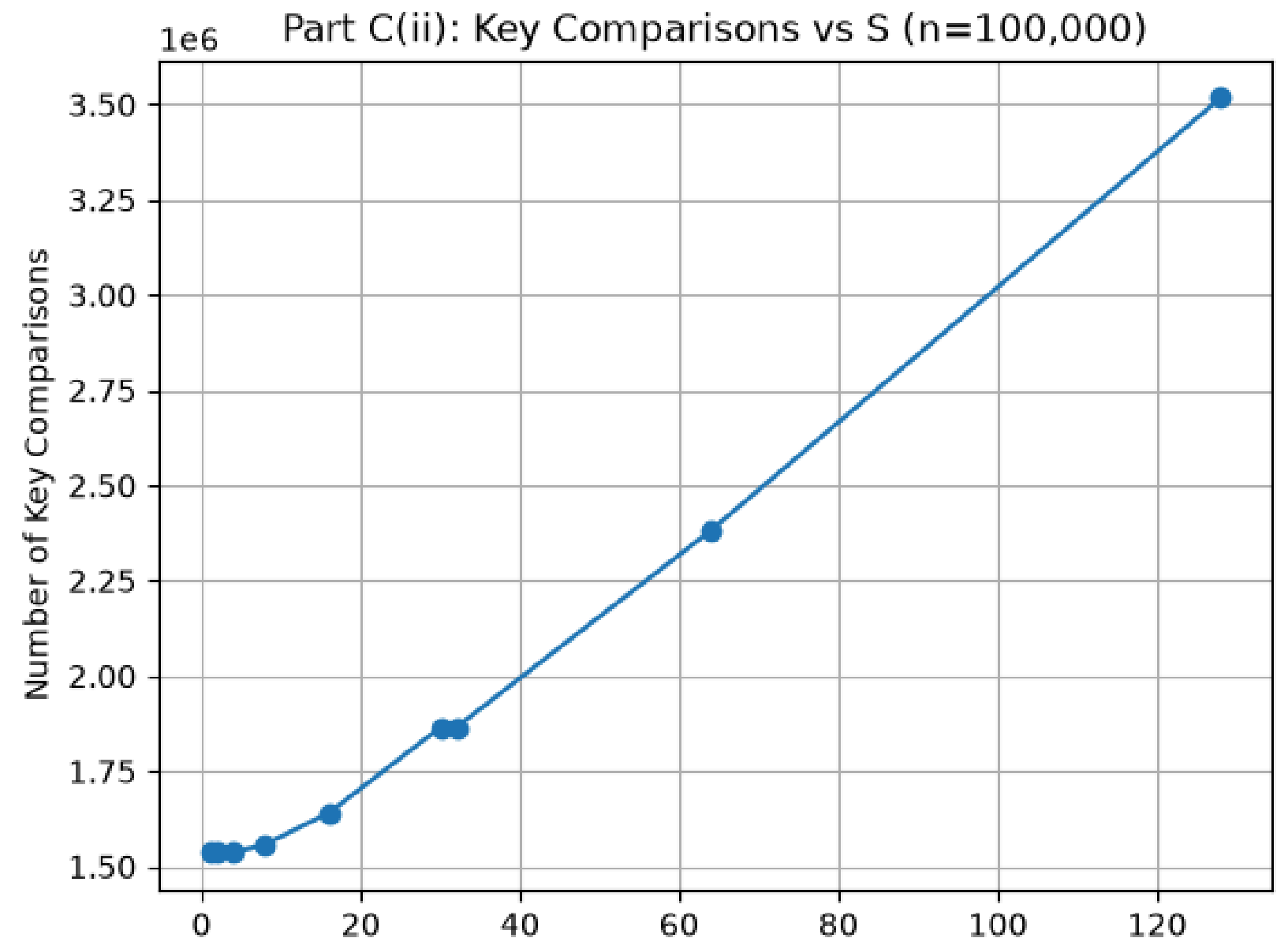
Small S

Algorithm will keep splitting longer due to more recursive calls and more smaller merge operations

Large S

Algorithm will stop the recursion earlier, letting the Insertion Sort do most of the work. Since Insertion Sort is inefficient when the inputs are larger, we would expect the number of key comparisons to increase when S becomes too large.

S=	1	comparisons=	1,536,324	CPU time=0.7656s
S=	2	comparisons=	1,536,324	CPU time=0.7031s
S=	4	comparisons=	1,536,902	CPU time=0.6406s
S=	8	comparisons=	1,558,060	CPU time=0.6250s
S=	16	comparisons=	1,640,762	CPU time=0.6250s
S=	30	comparisons=	1,865,125	CPU time=0.6406s
S=	32	comparisons=	1,865,125	CPU time=0.6562s
S=	64	comparisons=	2,382,507	CPU time=0.7812s
S=	128	comparisons=	3,517,613	CPU time=1.0469s



Observations: Comparing S=8 to S=1, when S=8, it actually ran faster while having more comparisons. This could be because key comparisons measure only 1 work but CPU time takes into considerations of recursion, function calls, array splitting, etc.

The empirical results support our expectations of when S becomes too large, number of comparisons will increase.

Moderate S values can still reduce CPU time by decreasing recursive overhead

C(iii): Finding the Optimal S

Set Up

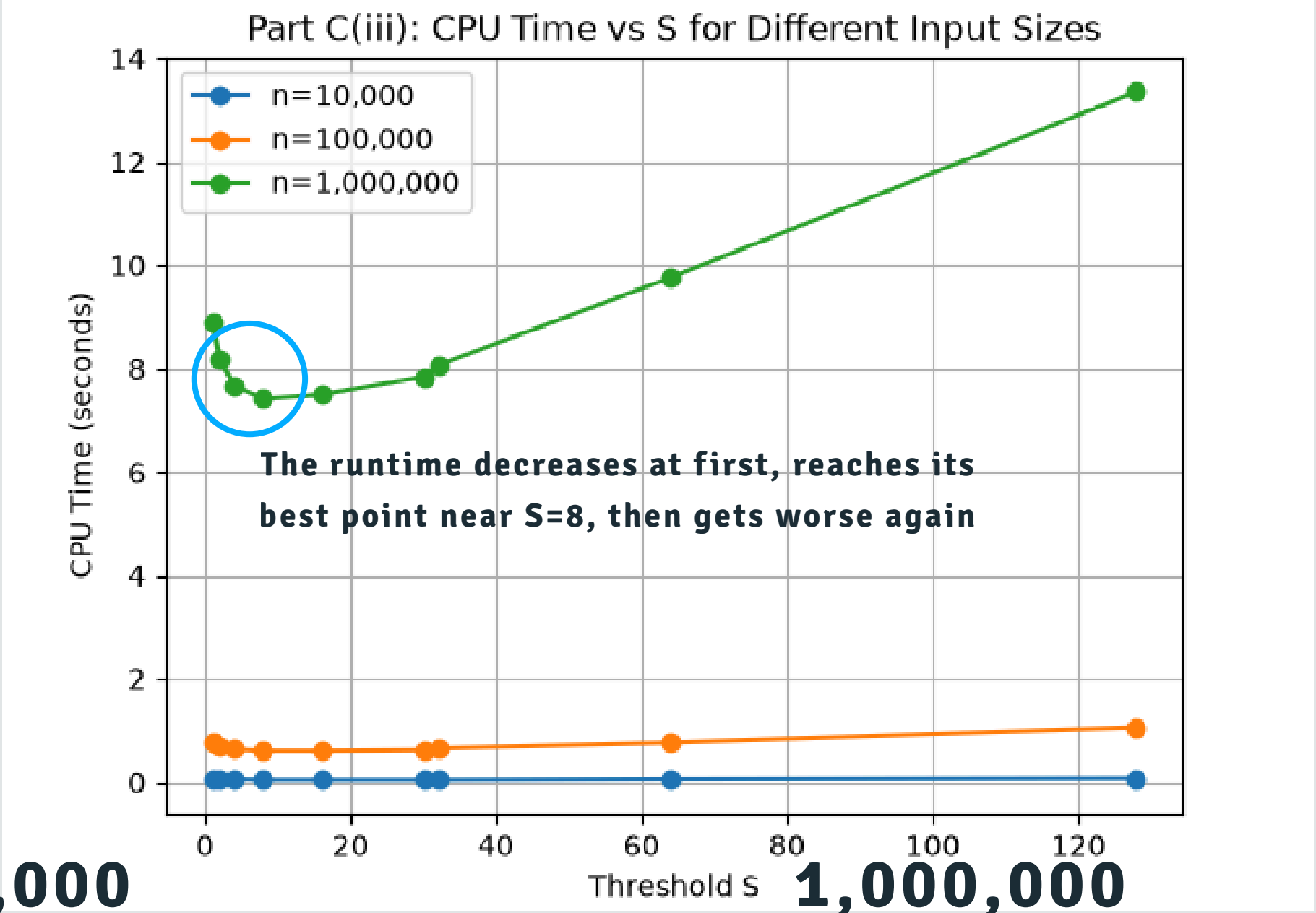
- 3 different input sizes: $n=10,000$ & $100,000$ & $1,000,000$
- Varying S : 1,2,4,8,16,30,32,64,128
- Measure: Number of key comparisons + CPU time

Theoretical Expectation

- Small $S \rightarrow$ more recursion overhead
- Large $S \rightarrow$ more Insertion Sort work
- Best S should lie somewhere in between

Why CPU time first decrease?: Alogrithm stops the Merge Sort earlier, reuslting in lesser recursive call $\rightarrow$ improves CPU time

Why CPU time increase again: The moment S becomes too large, Insertion Sort will be left with sorting larger chunks $\rightarrow$ both comparisions and CPU time will increase



10,000

100,000

1,000,000

S= 1	comparisons=	120,508	CPU time=0.0625s
S= 2	comparisons=	120,508	CPU time=0.0625s
S= 4	comparisons=	120,591	CPU time=0.0469s
S= 8	comparisons=	121,493	CPU time=0.0469s
S= 16	comparisons=	126,890	CPU time=0.0469s
S= 30	comparisons=	143,479	CPU time=0.0469s
S= 32	comparisons=	143,479	CPU time=0.0469s
S= 64	comparisons=	183,112	CPU time=0.0625s
S= 128	comparisons=	273,484	CPU time=0.0781s

S= 1	comparisons=	1,536,324	CPU time=0.7656s
S= 2	comparisons=	1,536,324	CPU time=0.7031s
S= 4	comparisons=	1,536,902	CPU time=0.6562s
S= 8	comparisons=	1,558,060	CPU time=0.6094s
S= 16	comparisons=	1,640,762	CPU time=0.6094s
S= 30	comparisons=	1,865,125	CPU time=0.6562s
S= 32	comparisons=	1,865,125	CPU time=0.6562s
S= 64	comparisons=	2,382,507	CPU time=0.7656s
S= 128	comparisons=	3,517,613	CPU time=1.0781s

S= 1	comparisons=	18,674,379	CPU time=8.8438s
S= 2	comparisons=	18,674,379	CPU time=8.1406s
S= 4	comparisons=	18,727,195	CPU time=7.7500s
S= 8	comparisons=	19,071,543	CPU time=7.6250s
S= 16	comparisons=	20,219,907	CPU time=7.5312s
S= 30	comparisons=	21,593,998	CPU time=7.8125s
S= 32	comparisons=	23,185,181	CPU time=8.1562s
S= 64	comparisons=	29,906,998	CPU time=9.7500s
S= 128	comparisons=	44,221,652	CPU time=13.2656s

D: Merge Sort Vs Hybrid Merge Sort

Set Up

- Input size: $n=10,000,000$
- Same dataset for both algorithms
- Hybrid threshold: $S=8$
- Compare: Key comparisons + CPU time

Theoretical Expectation

We expected the hybrid algorithm to have lesser overhead on small subarrays.
Having lesser recursion overhead could decrease the CPU time as well.

Original Merge Sort

```
Original Merge Sort  
Key comparisons: 220,100,416  
CPU time: 101.2031s
```

Hybrid Merge Sort

```
Hybrid Merge Sort  
Key comparisons: 221,111,817  
CPU time: 88.8438s
```

Comparison difference between Hybrid - Merge: 1,011,401 (0.46% more comparisons)

Time Saved: 12.3594s

CPU time reduction: 12.21%

Although Hybrid Merge Sort had more key comparisons, it was 12.21% faster because it reduced recursive overhead on small subarrays. It achieves better practical performance despite having more key comparisons.



Conclusion

- Hybrid preserves efficient scaling.

With a fixed threshold SS , the number of key comparisons increased in a way consistent with Merge Sort's expected $O(n \log n)$ behaviour as n grew

- Thresh S creates a trade off

Small S : more Merge Sort recursion & overhead

Large S : Insertion Sort handles larger subarrays and will have more comparison

- Lower Comparisions $\neq$ Lower CPU time

CPU time is affected by: recursive calls, merging, splitting, etc and not just comparing values

- Moderate threshold gave the best balance ($S=8$).

- Hybrid Merge Sort can improve practical performance

Improved practical runtime not by reducing the number of comparisons, but by reducing amount of recursive overhead.





Thank You

Any Questions?

